УГНТУ • ЦЕНТР ТОП ИТ • 09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА

Программирование и прототипирование ИТ-решений

Управляющие конструкции: if–elif–else, match-case, валидация входных данных

ЛЕКЦИЯ 4

Ахметов Ильнур Вазирович

План лекции

1. Повторение: логические выражения и таблицы истинности.
2. Ветвления: if, if-else, if-elif-else, вложенные конструкции.
3. Оператор match-case (Python 3.10+).
4. Валидация входных данных: тип, диапазон, защитные приёмы.
5. Практикумы и кейсы из нефтегазовой отрасли.

В конце — вопросы для закрепления (по одному на слайд).

ПОВТОРЕНИЕ

Логические выражения: что мы знаем

- bool — два значения: True и False.
- Операторы сравнения: ==, !=, <, >, <=, >=.
- Логические: and (оба), or (хоть один), not (инверсия).
- Приоритет: not > and > or.
- Результат выражения с оператором сравнения — всегда bool.

консоль Python

```
>>> p = 25.0   # пластовое, МПа
>>> z = 18.5   # забойное, МПа
>>> p > z
True
>>> 5 <= z <= 30   # в диапазоне
True
>>> (p > z) and (z > 0)
True
```

Таблицы истинности: and, or, not

and True and True → True True and False → False

False and True → False False and False → False

or True or True → True True or False → True

False or True → True False or False → False

not not True → False not False → True

and — «оба» or — «хотя бы один» not — «наоборот»

консоль Python

```
>>> в_работе = True
>>> давление_ок = False

>>> в_работе and давление_ок
False    # оба должны быть True

>>> в_работе or давление_ок
True     # хотя бы одно True

>>> not в_работе
False
```

Приоритет логических операторов

- Приоритет (от высшего к низшему):
- 1. сравнения (<, >, ==, !=, ...)
- 2. not
- 3. and
- 4. or
- True or False and False → True (and сначала!)
- (True or False) and False → False
- Правило: ставьте скобки — код читается однозначно.

консоль Python

```
>>> True or False and False
True      # and раньше: F and F = F,
           # потом True or F = True

>>> (True or False) and False
False     # скобки: T or F = T,
           # потом T and F = False
```

Короткое замыкание (short-circuit)

- and: если первый False — второй не вычисляется.
- or: если первый True — второй не вычисляется.
- Зачем: экономия; защита от вызова, если первый аргумент «плохой».
- x != 0 and (100 / x) > 10 — деление не выполнится при x=0.
- not тестируется первым (приоритет выше and).

консоль Python

```
# защита от деления на ноль
>>> x = 0
>>> x != 0 and (100 / x) > 10
False    # (100/x) не вычислялось!

>>> def check(): print('check'); return True
>>> False and check()
False    # check() не вызвалась
```

Практика: проверка давления в норме

- Задача: давление должно быть в диапазоне [5; 30] МПа,
- и режим добычи активен (активный=True).
- Решение: цепочка сравнений + and.
- в_норме = (5 <= p <= 30) and активный
- Такой bool-флаг прямо идёт в if — следующая тема.

код

```
p = 18.5          # МПа
активный = True

в_норме = (5 <= p <= 30) and активный
# True

авария = p < 5 or p > 30
# False

if в_норме and not авария:
    print("Скважина в норме")
```

ПРАКТИКА

Составные условия для диагностики скважины

- Три показателя: давление P, дебит Q, обводнённость W.
- Норма: 5<=P<=30, Q>0, W<0.90.
- Составное условие с and объединяет все проверки.
- Формируем флаг «нужно обслуживание»: P<5 or Q==0 or W>=0.90.

код

```
P, Q, W = 18.5, 12.0, 0.25

норма = (5<=P<=30) and (Q>0) and (W<0.90)
# True — все показатели в порядке

тревога = (P<5) or (Q==0) or (W>=0.90)
# False — тревоги нет

if норма and not тревога:
    print("Скважина работает штатно")
```

if: синтаксис и отступы

- if <условие>:
- <блок> — 4 пробела (СТАНДАРТ!)
- Блок выполняется, если условие истинно.
- После блока код продолжается на прежнем уровне отступа.
- Отступ (indentation) — часть синтаксиса, а не украшение.
- Tab меняется на 4 пробела! Настройте VS Code.

код

```
p = 18.5

if p > 20:
    print("Давление повышено")

if p < 5:
    print("Давление критически низкое")
    авария = True

print("Проверка завершена")
```

if-else: две ветки

- if <условие>:
- <блок если True>
- else:
- <блок если False>
- Выполнится РОВНО ОДНА из двух веток.
- Пример: скважина в работе или остановлена.

код

```
p = 18.5

if p >= 5:
    статус = "в работе"
else:
    статус = "остановлена — нет давления"

print(f"Скважина: {статус}")
# Скважина: в работе

# А что при p=18.5? -> в работе
```

if-elif-else: множественный выбор

- elif — сокращение от else if; можно несколько.
- Проверяет условия последовательно; ПЕРВОЕ истинное — выполняется.
- else — ЛОВИТ ВСЁ ОСТАЛЬНОЕ (если ни одно не подошло).
- Порядок ВАЖЕН: частное условие — перед общим.

код

```
Q = 65  # дебит, т/сут

if Q <= 0:
    режим = "остановлена"
elif Q < 10:
    режим = "малодебитная"
elif Q < 100:
    режим = "норма"
elif Q < 500:
    режим = "высокодебитная"
else:
    режим = "фонтанирование"

print(режим)  # "норма"
```

Вложенные if

- Внутри блока if может быть ещё один if.
- Полезно для уточняющей проверки.
- Глубина > 2–3 → задумайтесь об упрощении.
- Правило: внутренний if → уточнение внешнего условия.

код

```
p = 22.0    # МПа
Q = 85      # т/сут

if p >= 5:            # давление есть
    if Q > 0:         # дебит есть
        статус = "в работе"
    else:
        статус = "нет притока"
else:
    статус = "нет давления"

print(статус)  # "в работе"
```

Практика: классификация дебита скважины

- Задача: по суточному дебиту присвоить категорию.
- Категории: остановлена, малодебитная, норма, высокодебитная, фонтан.
- Используем if-elif-else; порядок — от меньшего к большему.
- Границы: ≤0, <10, <100, <500, остальное.

код

```
Q = float(input("Дебит, т/сут: "))

if Q <= 0:
    cat = "остановлена"
elif Q < 10:
    cat = "малодебитная"
elif Q < 100:
    cat = "норма"
elif Q < 500:
    cat = "высокодебитная"
else:
    cat = "фонтанирование"

print(f"Категория: {cat}")
```

Практика: выбор режима насоса

- Задача: по давлению выбрать режим ЭЦН (электроцентробежный насос).
- P < 5 → аварийная остановка;
- 5 ≤ P < 15 → низкая нагрузка, снизить частоту;
- 15 ≤ P ≤ 25 → штатный режим;
- P > 25 → перегрузка, поднять предохранительный клапан.

код

```
P = 22.5  # МПа

if P < 5:
    режим = "АВСТ: аварийная остановка"
elif 5 <= P < 15:
    режим = "снизить частоту"
elif 15 <= P <= 25:
    режим = "штатный режим"
else:  # P > 25
    режим = "перегрузка: открыть клапан"

print(f"Насос: {режим}")
```

Тернарный оператор: x if условие else y

- Компактная запись if-else В ОДНУ СТРОКУ.
- x if условие else y
- Удобно для присваивания; не злоупотребляйте (читаемость).
- статус = "в работе" if Q > 0 else "остановлена"

консоль Python

```
>>> Q = 85
>>> status = "в работе" if Q > 0 else "стоп"
>>> status
'в работе'

>>> p = 4.2
>>> flag = "норма" if 5<=p<=30 else "вне
диапазона"
>>> flag
'вне диапазона'
```

Практика: надбавка к дебиту

- После ГТМ (геолого-технического мероприятия) ожидается прирост.
- Если дебит > 100 т/сут — надбавка 10%, иначе 5%.
- Тернарный оператор в одну строку.

код

```
Q = 120     # т/сут после ГТМ

k = 1.10 if Q > 100 else 1.05

Q_adj = Q * k

print(f"Скорректированный дебит: {Q_adj:.1f}
т/сут")
# Скорректированный дебит: 132.0 т/сут
```

match-case: синтаксис (Python 3.10+)

- match <значение>:
- case <шаблон>:
- <блок>
- case <другой>:
- <блок>
- Работает как switch в C/Java, но мощнее — сопоставление с шаблоном.
- Первый совпавший case — выполняется; _ — «всё остальное».

код

```
code = 2

match code:
    case 1:
        mode = "добыча"
    case 2:
        mode = "нагнетание"
    case 3:
        mode = "остановлена"
    case _:
        mode = "неизвестный код"

print(mode)  # "нагнетание"
```

match с литералами

- Можно сопоставлять строки, числа, True/False/None.
- case 0: — точное совпадение числа; case "газ": — строки.
- case _: — универсальный «любой другой» (wildcard).

консоль Python

```
def fluid_type(name):
    match name:
        case "нефть": return "oil"
        case "газ":  return "gas"
        case "вода":  return "water"
        case _:       return "unknown"

>>> fluid_type('газ')
'gas'
```

match с OR-шаблонами (|)

- Символ | — «или»: несколько значений, один блок.
- case "нефть" | "газ": — для обоих один код.
- Удобно для группировки похожих случаев.

код

```
well_type = "добывающая"

match well_type:
    case "добывающая" | "нагнетательная":
        category = "эксплуатационная"
    case "разведочная" | "поисковая":
        category = "геологическая"
    case _:
        category = "прочая"

print(category)  # "эксплуатационная"
```

match с последовательностями (tuple/list)

- Можно сопоставлять структуру: (x, y) — кортеж из двух.
- case (0, 0): — начало координат; case (x, 0): — на оси X.
- Переменные в шаблоне связываются со значениями.

код

```
def classify_point(pt):
    match pt:
        case (0, 0):
            return "origin"
        case (x, 0):
            return f"x-axis at {x}"
        case (0, y):
            return f"y-axis at {y}"
        case (x, y):
            return f"point ({x}, {y})"

>>> classify_point((0, 5))
'y-axis at 5'
```

match с guard-условиями (if после case)

- case <шаблон> if <условие>: — guard (защитное условие).
- Шаблон совпал И условие истинно → блок выполнен.
- Позволяет уточнить совпадение диапазоном/доп. проверкой.

код

```
def pump_mode(pressure, flow):
    match (pressure, flow):
        case (p, q) if p < 5:
            return "STOP: low pressure"
        case (p, q) if q == 0:
            return "STOP: no flow"
        case (p, q) if p > 25 and q > 300:
            return "OVERLOAD"
        case (p, q):
            return "NORMAL"

>>> pump_mode(28, 350)
'OVERLOAD'
```

match с шаблонами словарей

- Можно сопоставлять ключи словаря.
- case {"тип": "нефть", "дебит": q}: — проверяет ключ «тип» и извлекает «дебит».
- Порядок ключей в словаре не важен для match.

код

```
def parse_well(data):
    match data:
        case {"тип": "нефтяная", "Q": q}:
            return f"Нефть: {q} т/сут"
        case {"тип": "газовая", "Q": q}:
            return f"Газ: {q} тыс.м³/сут"
        case {"тип": "нагнетательная"}:
            return "Нагнетание"
        case _:
            return "Неизвестный тип"
```

Практика: match по коду скважины

- Задача: код скважины вида «К-12-НК» или «К-12-Д».
- Последняя часть — назначение: НК, Д, Н, В, Г.
- Распознаём через match с литералами и выводим расшифровку.

код

```
code = "К-12-Д"
parts = code.split("-")
match parts[-1]:
    case "НК":   desc = "нефтяная кустовая"
    case "Д":    desc = "добывающая"
    case "Н":    desc = "нагнетательная"
    case "В":    desc = "водозаборная"
    case "Г":    desc = "газовая"
    case _:      desc = "неизвестное назначение"

print(f"{code} → {desc}")
# К-12-Д → добывающая
```

ПРАКТИКА

match vs if: когда что использовать

if-elif лучше для диапазонов (x < 10, 10 <= x < 50).

match лучше для точных значений (коды, строки, структуры).

match читается чище при переборе 4+ вариантов по одному полю.

if гибче — любое условие, не только равенство.

В одном проекте могут соседствовать оба — это нормально.

код

```
# if — для диапазонов
if Q < 10:      cat = 'low'
elif Q < 100:   cat = 'mid'
else:           cat = 'high'

# match — для точных кодов
match well_code:
    case "Д":  mode = "добыча"
    case "Н":  mode = "нагнетание"
    case _:    mode = "?"
```

Валидация: зачем и как

- Валидация = проверка данных до использования.
- Без неё программа падает; с ней — сообщает и даёт исправить.
- Три уровня: тип, диапазон, бизнес-правила.
- Лучше не пустить неверные данные, чем упасть на полпути.

код

```
# без валидации
age = int(input("Возраст: "))
# ввод 'abc' → ValueError → крах

# с валидацией
while True:
    s = input("Возраст: ")
    if s.isdigit():
        age = int(s); break
    print("Введите число!")
```

Проверка типа: isinstance()

- isinstance(x, тип) — True, если x этого типа или подкласса.
- isinstance(3.14, float) → True
- isinstance(42, (int, float)) → True (кортеж — «или»).
- Лучше, чем type(x) == int — работает с наследованием.

консоль Python

```
>>> isinstance(42, int)
True
>>> isinstance(3.14, (int, float))
True     # int ИЛИ float
>>> isinstance('12', int)
False
>>> isinstance(None, int)
False
```

Проверка диапазона значений

- После проверки типа — проверка допустимых границ.
- min_val <= x <= max_val — компактная запись.
- Для каждого параметра — свои границы (давление, дебит, обводнённость).
- Границы должны быть ОСМЫСЛЕННЫМИ (физика + нормативы).

код

```
def validate_pressure(p):
    if not isinstance(p, (int, float)):
        return False, "не число"
    if not (0 <= p <= 50):
        return False, "вне диапазона [0; 50] МПа"
    return True, "OK"

>>> validate_pressure(18.5)
(True, 'OK')
>>> validate_pressure(-5)
(False, 'вне диапазона [0; 50] МПа')
```

Проверка на None и пустые значения

- None, "", [], {} — «пустые» значения, которые могут сломать логику.
- x is None — проверка на отсутствие значения.
- if not x: — True для None, 0, "", [], {} (осторожно с 0!).
- Лучше явно: if x is None или if x == "".

код

```
def safe_divide(a, b):
    if b is None or b == 0:
        return "ошибка: b is None or 0"
    return a / b

>>> safe_divide(100, 0)
'ошибка: b is None or 0'

# Осторожно с not!
x = 0
if not x:        # True! 0 считается falsy
    print("x пустой или ноль")
```

Ранний возврат (guard clauses)

- Вместо вложенных if — проверки в начале функции.
- Если условие НЕ выполнено → return или raise.
- Код становится «плоским», читается легче.
- «Сначала отсеки плохое, потом делай дело».

код

```
def calc_rate(oil, water, days):
    # guard-ы
    if days is None or days <= 0:
        return "Ошибка: days"
    if oil < 0 or water < 0:
        return "Ошибка: отриц. значения"

    total = oil + water
    if total == 0:
        return "Нет добычи"

    # основная логика
    return total / days
```

Сбор ошибок: список вместо первого же return

- Иногда нужно сообщить ВСЕ ошибки сразу, а не первую.
- Собираем их в список errors = [].
- В конце: if errors: → return/imprimer все ошибки.
- Пользователь видит полную картину и исправляет за один раз.

код

```
def validate_well(p, q, wcut):
    errors = []
    if not (0 <= p <= 50):
        errors.append("давление вне [0;50]")
    if q < 0:
        errors.append("дебит отрицательный")
    if not (0 <= wcut <= 1):
        errors.append("обводнённость не в [0;1]")

    if errors:
        return False, errors
    return True, None
```

Практика: валидатор параметров скважины

- Задача: три параметра — P (давление), Q (дебит), W (обводнённость).
- Проверить тип (float/int), диапазон, бизнес-правило:
- если P > 0 → Q должен быть > 0.
- Собрать все ошибки и вывести.

код

```
def validate(P, Q, W):
    errs = []
    if not isinstance(P,(int,float)) or not
(0<=P<=50):
        errs.append("P: [0;50] МПа")
    if not isinstance(Q,(int,float)) or Q < 0:
        errs.append("Q >= 0")
    if not isinstance(W,(int,float)) or not
(0<=W<=1):
        errs.append("W: [0;1]")
    if P > 0 and Q == 0:
        errs.append("Давление есть, дебита нет")
    return len(errs) == 0, errs
```

Практика: валидатор ввода с if-elif

- Защита от «дурака»: принимаем строку, проверяем, даём шанс исправить.
- Цикл while True + проверки + break при успехе.
- Каждая ошибка — понятное сообщение, а не голый ValueError.

код

```
def input_pressure():
    while True:
        s = input("Давление, МПа: ").strip()
        if s == "":
            print("Пустой ввод, повторите")
            continue
        if not s.replace('.','',1).isdigit():
            print("Введите число!")
            continue
        p = float(s)
        if not (0 <= p <= 50):
            print("Давление должно быть [0;50]")
            continue
        return p
```

КЕЙС

Классификатор режимов добычи скважины

- Сквозной кейс: на входе P и Q, на выходе — режим и рекомендация.
- Используем вложенные if / guard-ы для чёткой логики.
- Отсеиваем невалидные; классифицируем; выдаём действие.

код

```
def diagnose(P, Q):
    if not (0<=P<=50) or Q<0:
        return "невалидные данные"

    if P < 5 or Q == 0:
        return "АВАРИЯ: остановка"
    if P < 15:
        return "ПРЕДУПРЕЖДЕНИЕ: снизить частоту"
    if P > 25 and Q > 300:
        return "ПЕРЕГРУЗКА: открыть клапан"

    return "НОРМА"
```

КЕЙС

Полный валидатор параметров с if-elif

- Запрашиваем P, Q, W; проверяем каждое; сообщаем ошибки.
- Если хоть одна ошибка — не продолжаем, просим исправить всё.
- Иначе — классифицируем режим (предыдущий кейс).

код

```
def well_check():
    P = input_pressure()
    Q = float(input("Дебит: "))
    W = float(input("Обводнённость: "))

    ok, errs = validate(P, Q, W)
    if not ok:
        print("Ошибки:", errs)
        return

    result = diagnose(P, Q)
    print(f"Режим: {result}")
    print(f"Обводнённость: {W:.1%}")
```

Предскажите результат

- Не запуская, предскажите результат каждого выражения.

1. True and False or True
2. x = 15; if x>20: print("A"); elif x>10: print("B"); else: print("C")
3. x = 0; print("OK") if x>0 else print("NOK")
4. match 3: case 1|2: "A"; case _: "B"
5. not (5 > 3 and 2 < 1)
6. x = 5; if x: print("Yes")



Предскажите результат

- Не запуская, предскажите результат каждого выражения.

1. True and False or True
2. x = 15; if x>20: print("A"); elif x>10: print("B"); else: print("C")
3. x = 0; print("OK") if x>0 else print("NOK")
4. match 3: case 1|2: "A"; case _: "B"
5. not (5 > 3 and 2 < 1)
6. x = 5; if x: print("Yes")

ответы

```
1. True     (and: T and F = F, or: F or T = T)
2. "B"      (x>20→F, x>10→T)
3. "NOK"    (0→F, else-ветка)
4. "B"       (3 не 1 и не 2, wildcard _)
5. True     (внутри False, not → True)
6. "Yes"    (5 → truthy)
```

Найдите и исправьте ошибку

1. if x = 5: print("OK")
2. if Q > 0:

print("OK")

  print("ещё")

3. match x:

case 1 | 2 | 3:

print("OK")

4. if not isinstance(x, int)

|| x < 0:

return

5. if давление > 0 and дебит == 0:

print("Ошибка")

Подсказка: = vs ==, отступы, синтаксис, приоритет.



Найдите и исправьте ошибку

1. if x = 5: print("OK")
2. if Q > 0:

print("OK")

 print("ещё")

3. match x:

case 1 | 2 | 3:

print("OK")

4. if not isinstance(x, int)

|| x < 0:

return

5. if давление > 0 and дебит == 0:

print("Ошибка")

Подсказка: = vs ==, отступы, синтаксис, приоритет.

исправления

```
1. if x == 5:      # = → ==
2. добавить отступ:
    if Q > 0:
        print("OK")
        print("ещё")  # 4 пробела
3. case 1 | 2 | 3:  # убрать лишний пробел
   в остальном — OK
4. if not isinstance(x,int) or x<0:
   || → or
5. кириллица в именах — работает,
   но не по PEP 8. OK для прототипа.
```

Типичные ошибки и ловушки

= vs == if x = 5 → SyntaxError. В условии — всегда ==.

Отступы забыли отступ после if → IndentationError.

Порядок elif частное условие должно быть ПЕРЕД общим.

if not x x=0 тоже пройдёт; используйте x is None для проверки на None.

match: забыли _ без wildcard-а — MatchError для неучтённых значений.

Тернарник: читаемость не вкладывайте тернарник в тернарник.

Итоги лекции

1. Логические операторы and/or/not, таблицы истинности, короткое замыкание.
2. Ветвления: if, if-else, if-elif-else. Отступы — часть синтаксиса.
3. Тернарный оператор: x if условие else y.
4. match-case для сопоставления с шаблоном (литералы, последовательности, guard-ы).
5. Валидация: проверка типа, диапазона, ранний возврат, сбор ошибок.
6. Итоговый результат: пишем устойчивые программы, которые не падают на неверных данных.

Вопрос 1 из 6

Что выведет: True and False or True? Почему?

Вопрос 2 из 6

Чем if-elif-else отличается от цепочки if-if-if? Когда сработает elif, а когда — нет?

Вопрос 3 из 6

Что выведет: x=15; if x>20: print('A'); elif x>10: print('B')? Объясните почему не 'A'.

Вопрос 4 из 6

В чём преимущество match перед if-elif для обработки кодов скважин? А когда if лучше?

Вопрос 5 из 6

Как проверить, что давление задано (не None) и находится в [5; 30], одним составным условием?

Вопрос 6 из 6

Почему int(input()) может упасть с ValueError? Как защититься if-конструкцией?